

godot interface in general

godot is organized into three main sections

the left side, which holds your scene tree and wile manager

the center which hold your game preview and editor

the right side which holds your inspector and signals

nodes

nodes are the fundamental building block in godot, they hold properties, are usually a parent for/a child of other nodes, so they can have the properties of the parent class

they also hold specific functions accessible through signals and scripts

scenes

A collection of nodes with one main node

usually organized based on task where the order of the nodes on the tree, especially the order of which ones are children and which ones are parents are important

instances

Ways to copy existing scenes and have multiple of them in other scenes, where any edits to the main scene updates all instances

scripts

ways of extending the capability of a singular node

physics process runs 60 fps or lower

and

process runs as fast as possible

where physics should usually be applied in physics process

signals

ways of making code run only on specific events

rigid bodies

difference between kinematic, static, and rigid body

A kinematic body is a body that can collide but has to have every interaction be programmed in

a static body can collide and produce effects on other bodies but cannot move themselves

a rigid body can collide and also gets affected by external or internal forces only

area2d, also collision bodies, and how they can have physics on them

area 2d specifically can manipulate/replace the physics attribute of a rigid body touching the area2d

physics materials and how they manipulate the physics of an object

benefits of using rigid bodies over kinematics

gives you a bunch of complex behavior for free

cons

snappy controls are harder, and physics is more expensive

physics bodies and how godot calculates their movement

godot has a thing called the physics server which calculates all of the movement that will occur in a given frame, it happens before process usually

if you try to manipulate their movement, it becomes less reliable, use integrate forces to be able to set values cleanly,

integrate forces and how to manipulate physics in real time

also allows access of complex collision data